

Lecture 15:

Multidimensional Data with NumPy



INFO 1100: Introduction to Programming

a lot of data we might be interested in is
multidimensional

a lot of data we might be interested in is
multidimensional

gradebooks

*(students x
assignments)*

a lot of data we might be interested in is
multidimensional

gradebooks

*(students x
assignments)*

blood tests

(patients x tests)

a lot of data we might be interested in is
multidimensional

gradebooks

*(students x
assignments)*

images

*(pixels x pixels x
rgb value)*

blood tests

(patients x tests)

a lot of data we might be interested in is
multidimensional

gradebooks

*(students x
assignments)*

blood tests

(patients x tests)

images

*(pixels x pixels x
rgb value)*

temperature
readings

(day x time)

multidimensional lists?

multidimensional lists?

a good option!

multidimensional lists?

a good option!

but...

multidimensional lists?

a good option!

but...

difficult to
manage



multidimensional lists?

a good option!

but...

difficult to
manage

slow



NumPy

NumPy

a Python package

NumPy

a Python package



a collection of
modules

NumPy

a Python package

lets us use NumPy arrays

NumPy arrays

NumPy arrays

What are they?

NumPy arrays

store a sequence of numbers

NumPy arrays

store a sequence of numbers

can be indexed and sliced

NumPy arrays

store a sequence of numbers

can be indexed and sliced

like **lists**!

NumPy arrays

store a sequence of numbers

can be indexed and sliced

but...

NumPy arrays

store a sequence of numbers

can be indexed and sliced

make it very easy to
access data by
dimension

NumPy arrays

store a sequence of numbers

can be indexed and sliced

make it very easy to
access data by
dimension

highly optimized for
mathematical
operations on contents

NumPy arrays

store a sequence of numbers

can be indexed and sliced

make it very easy to
access data by
dimension

vectorized for
mathematical
operations on contents

NumPy arrays

store a sequence of numbers

can be *indexed* and *sliced*

must be

homogeneous

let's write some code!

```
import numpy as np
```

```
import numpy as np
```

convention

```
import numpy as np
```

```
import numpy as np
```

we can create NumPy arrays
from lists using `np.array()`

```
import numpy as np
```

```
arr = np.array([3.8, 4.0, 6.2])
```

```
import numpy as np
```

```
arr = np.array([3.8, 4.0, 6.2])
```

```
import numpy as np
```

```
arr = np.array([3.8, 4.0, 6.2])
```

```
import numpy as np
```

```
arr = np.array([3.8, 4.0, 6.2])
```

3.8	4.0	6.2
-----	-----	-----

```
import numpy as np
```

```
arr = np.array([3.8, 4.0, 6.2])
```

indexed like a list

```
import numpy as np

arr = np.array([3.8, 4.0, 6.2])
print(arr[1])
```

```
import numpy as np
```

```
arr = np.array([3.8, 4.0, 6.2])
```

```
print(arr[1]) → 4.0
```

```
import numpy as np

arr = np.array([3.8, 4.0, 6.2])
print(arr[1])
print(arr[-1])
```

```
import numpy as np
```

```
arr = np.array([3.8, 4.0, 6.2])
```

```
print(arr[1])
```

```
print(arr[-1]) → 6.2
```

```
import numpy as np

arr = np.array([3.8, 4.0, 6.2])
print(arr[1])
print(arr[-1])
```

sliced like a list

```
import numpy as np

arr = np.array([3.8, 4.0, 6.2])
print(arr[1])
print(arr[-1])
print(arr[1:3])
```

```
import numpy as np

arr = np.array([3.8, 4.0, 6.2])
print(arr[1])
print(arr[-1])
print(arr[1:3]) → [4. 6.2]
```

```
import numpy as np

arr = np.array([3.8, 4.0, 6.2])
print(arr[1])
print(arr[-1])
print(arr[1:3])
```

dimensionality: the # of indices required
to specify an individual entry

```
import numpy as np

arr = np.array([3.8, 4.0, 6.2])
print(arr[1])
print(arr[-1])
print(arr[1:3])
```

axes: the # of indices required to specify
an individual entry

```
import numpy as np

arr = np.array([3.8, 4.0, 6.2])
print(arr[1])
print(arr[-1])
print(arr[1:3])
```

arr is one-dimensional

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

arr2 is two-dimensional

```
import numpy as np

arr2 = np.array([[87, 92],
                  [67, 52],
                  [94, 91]])
```

87	92
67	52
94	91

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])
```

we can index
both axes

87	92
67	52
94	91

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

	0	1
0	87	92
1	67	52
2	94	91

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

		← axis-1 →	
		0	1
← axis-0 ↓	0	87	92
	1	67	52
	2	94	91

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

then we use
one index per
axis

		← axis-1 →	
		0	1
← axis-0 ↓	0	87	92
	1	67	52
	2	94	91

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])

print(arr2[1, 0])
```

		← axis-1 →	
		0	1
← axis-0 ↓	0	87	92
	1	67	52
	2	94	91

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])

print(arr2[1, 0])
```

		← axis-1 →	
		0	1
← axis-0 ↓	0	87	92
	1	67	52
	2	94	91

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])

print(arr2[1, 0])
```

← axis-0 ↓		← axis-1 →	
		0	1
0		87	92
1		67	52
2		94	91

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])

print(arr2[1, 0])
```

		← axis-1 →	
		0	1
← axis-0 ↓	0	87	92
	1	67	52
	2	94	91

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])

print(arr2[1, 0]) → 67
```

		- axis-1 →	
		0	1
← axis-0 -	0	87	92
	1	67	52
	2	94	91

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

we can also
slice each axis
separately

← axis-0 ↓		← axis-1 →	
		0	1
0	0	87	92
	1	67	52
	2	94	91

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])

print(arr2[0:3, 1])
```

		← axis-1 →	
		0	1
← axis-0 ↓	0	87	92
	1	67	52
	2	94	91

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])

print(arr2[0:3, 1])
```

		← axis-1 →	
		0	1
← axis-0 ↓	0	87	92
	1	67	52
	2	94	91

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])

print(arr2[0:3, 1])
```

		← axis-1 →	
		0	1
← axis-0 ↓	0	87	92
	1	67	52
	2	94	91

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])

print(arr2[0:3, 1])
```

		- axis-1 →	
		0	1
← axis-0 -	0	87	92
	1	67	52
	2	94	91

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

```
print(arr2[0:3, 1])
```



[92 52 91]

		- axis-1 →	
		0	1
← axis-0 -	0	87	92
	1	67	52
	2	94	91

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

giving fewer
indices than axes
auto-fills slices

		← axis-1 →	
		0	1
← axis-0 ↓	0	87	92
	1	67	52
	2	94	91

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

```
print(arr2[1])
```

		← axis-1 →	
		0	1
← axis-0 ↓	0	87	92
	1	67	52
	2	94	91

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

```
print(arr2[1])
```



```
arr2[1, :]
```

		← axis-1 →	
		0	1
← axis-0 ↓	0	87	92
	1	67	52
	2	94	91

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

```
print(arr2[1])
```



```
arr2[1, :]
```

		- axis-1 →	
		0	1
← axis-0 -	0	87	92
	1	67	52
	2	94	91

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

```
print(arr2[1])
```



```
arr2[1, :]
```

		- axis-1 →	
		0	1
← axis-0 -	0	87	92
	1	67	52
	2	94	91

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

```
print(arr2[1])
```

↓
`arr2[1, :]`

← axis-0 ↓		← axis-1 →	
		0	1
0		87	92
1		67	52
2		94	91

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

```
print(arr2[1]) → [67 52]
```



```
arr2[1, :]
```

		- axis-1 →	
		0	1
← axis-0 -	0	87	92
	1	67	52
	2	94	91

arrays have many interesting properties
we can access

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])

print(arr2.shape) → (3, 2)
```

`.shape` finds the shape of a NumPy array

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

```
print(arr2.shape) → (3, 2)
```

↑

rows

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

```
print(arr2.shape) → (3, 2)
```

↑
cols

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])

print(arr2.shape)
print(arr2.ndim) → 2
```

`.ndim` finds the # dimensions

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])

print(arr2.shape)
print(arr2.ndim)
print(arr2.size) → 6
```

`.size` finds the # items

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])

print(arr2.shape)
print(arr2.ndim)
print(arr2.size)
print(arr2.dtype) → 'int64'
```

`.dtype` finds the data type of the elements

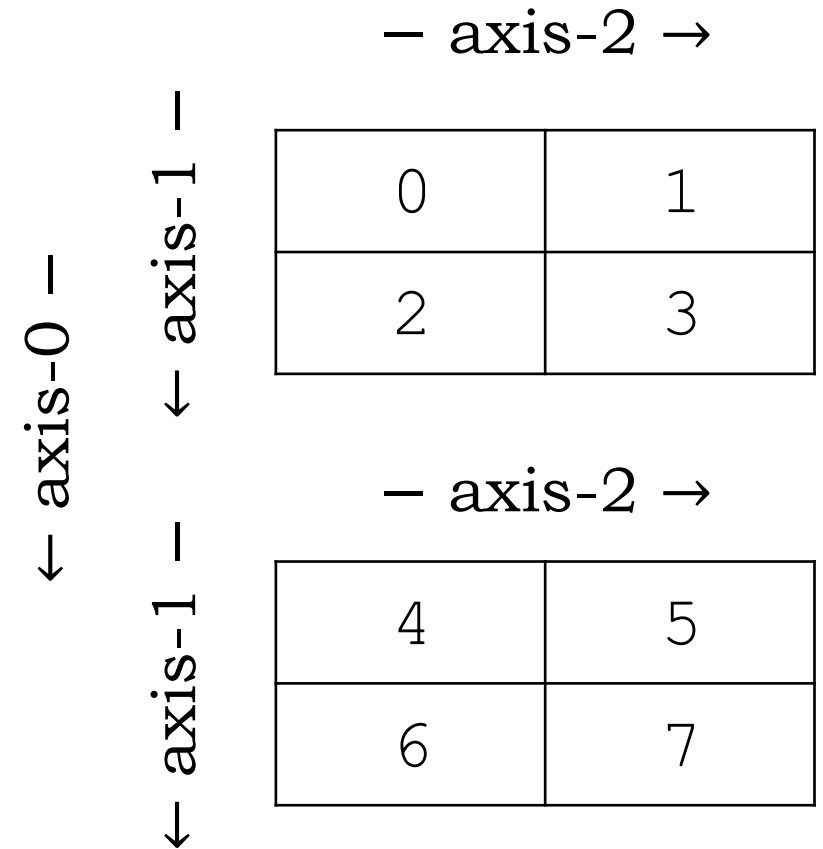
you can expand to as many dimensions
as your data has

```
import numpy as np
```

```
arr2 = np.array([[0, 1],
                  [2, 3],
                  [4, 5],
                  [6, 7]])
```

```
import numpy as np
```

```
arr2 = np.array([[0, 1],  
                 [2, 3],  
                 [4, 5],  
                 [6, 7]])
```



there are many functions that allow us to
make pre-filled arrays

there are many functions that allow us to
make pre-filled arrays

explore them on your own!

we can also do a lot of efficient operations
on arrays

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

mathematical operations are applied to
the whole array

```
import numpy as np

arr2 = np.array([[87, 92],
                  [67, 52],
                  [94, 91]])

print(arr2 + 2)
```

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

```
print(arr2 + 2)
```



*[[89 94]
 [69 54]
 [96 93]]*

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])
```

NumPy has a large number of built-in
unary functions

```
import numpy as np

arr2 = np.array([[87, 92],
                  [67, 52],
                  [94, 91]])

print(np.sqrt(arr2))
```

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

```
print(np.sqrt(arr2))
```



```
[[9.32737905  9.59166305]  
 [8.18535277  7.21110255]  
 [9.69535971  9.53939201]]
```

```
import numpy as np
```

NumPy also has a large number of built-in **binary** functions

```
import numpy as np
```

```
arr1 = np.array([1, 2, 3, 4])
```

```
arr2 = np.array([4, 6, 7, 8])
```

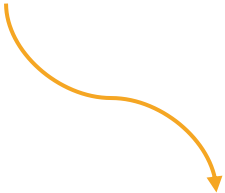
```
print(np.add(arr1, arr2))
```

```
import numpy as np
```

```
arr1 = np.array([1, 2, 3, 4])
```

```
arr2 = np.array([4, 6, 7, 8])
```

```
print(np.add(arr1, arr2))
```



[5, 8, 10, 12]

```
import numpy as np

arr1 = np.array([1, 2, 3, 4])
arr2 = np.array([4, 6, 7, 8])
print(np.add(arr1, arr2))
```

again, we can often use the Python
operator in place of the NumPy function

```
import numpy as np
```

```
arr1 = np.array([1, 2, 3, 4])
```

```
arr2 = np.array([4, 6, 7, 8])
```

```
print(arr1 + arr2)
```

```
import numpy as np
```

```
arr1 = np.array([1, 2, 3, 4])
```

```
arr2 = np.array([4, 6, 7, 8])
```

```
print(arr1 + arr2)
```



[5, 8, 10, 12]

```
import numpy as np
```

```
arr1 = np.array([1, 2, 3, 4])
```

```
arr2 = np.array([4, 6, 7, 8])
```

```
print(arr1 + arr2)
```

there are many, many of these functions!



[5, 8, 10, 12]

```
import numpy as np
```

```
arr1 = np.array([1, 2, 3, 4])
```

```
arr2 = np.array([4, 6, 7, 8])
```

```
print(arr1 + arr2)
```

there are many, many of these functions!

again, I encourage you to explore them on your own

[5, 8, 10, 12]

many functions also take an array and
return a single value

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

```
import numpy as np

arr2 = np.array([[87, 92],
                  [67, 52],
                  [94, 91]])

print(np.mean(arr2))
```

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])

print(np.mean(arr2)) → 80.5
```

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])

print(np.mean(arr2))
```

but what if we just want to average the
rows?

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])

print(np.mean(arr2))
```

but what if we just want to average the
rows? or the columns?

```
import numpy as np

arr2 = np.array([[87, 92],
                  [67, 52],
                  [94, 91]])

print(np.mean(arr2))
```

we can specify the **axis**!

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

```
print(np.mean(arr2, axis=0))
```

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

```
print(np.mean(arr2, axis=0))
```



[82.66666667, 78.33333333]

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])

print(np.mean(arr2, axis=1))
```

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

```
print(np.mean(arr2, axis=1))
```



[89.5, 59.5, 92.5]

```
import numpy as np

arr2 = np.array([[87, 92],
                 [67, 52],
                 [94, 91]])

print(np.mean(arr2, axis=1))
```

the axis you pass to the function is the
one that disappears!

```
import numpy as np
```

```
arr2 = np.array([[87, 92],  
                 [67, 52],  
                 [94, 91]])
```

again, there are many similar NumPy functions!

```
print(np.mean(arr2, axis=1))
```

the axis you pass to the function is the one that disappears!

so...

so...

why are we learning this again?

SO...

why are we learning this again?

thinking about
multidimensional
data

SO...

why are we learning this again?

thinking about
multidimensional
data

a useful tool I want
you to know for the
future

Example 1

```
import numpy as np

grades = np.array([[93, 95, 70],
                   [84, 100, 88],
                   [99, 87, 60]])

rows, cols = grades.shape
for c in range(cols):
    col = grades[:, c]
    print(np.max(col), np.min(col))
```

Example 2

```
def summarize(arr):  
    avg = np.mean(arr)  
    if avg >= 50:  
        return "pass"  
    else:  
        return "fail"  
  
scores = np.array([88, 92, 79, 95])  
print(summarize(scores))  
print(np.mean(scores))
```

Example 3

```
data = np.array([[25, 60],
                  [30, 72],
                  [35, 66]])
labels = ['age', 'weight']
result = {}
for i in range(len(labels)):
    result[labels[i]] = np.mean(data[:, i])
print(result)
```

Example 4

```
def hardest_exam(grades):  
    exam_avgs = np.mean(grades, axis=0)  
    return np.min(exam_avgs)  
  
grades = np.array([[90, 60, 80],  
                   [80, 50, 100],  
                   [70, 40, 90]])  
print(hardest_exam(grades))
```

Example 5

```
scores = np.array([[8, 7, 9],
                  [10, 6, 8],
                  [7, 9, 10]])

per_student = np.sum(scores, axis=1)
per_question = np.sum(scores, axis=0)
questions = ["Q1", "Q2", "Q3"]
totals = {}
for i in range(len(questions)):
    totals[questions[i]] = per_question[i]
print(per_student)
print(totals)
```

Questions?

(... I'll wait for 2)

For tomorrow:

(07/23 @ 11:30am)

- Lab 15
- Readings
Think Python –
7.2, 8.6, 13.1-13.2
“Working with Files”
- GRQs